

潜在変数からの RSRP エピソード生成

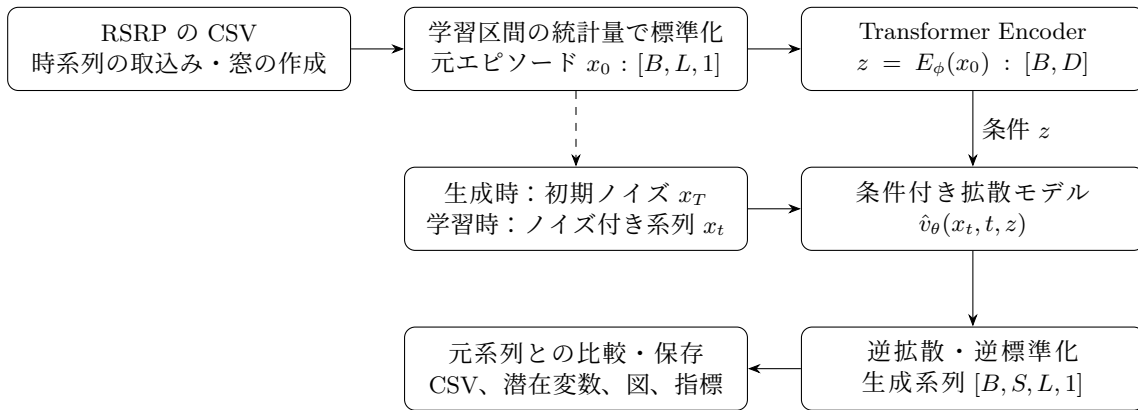
EpisodicDT-small：プログラム説明資料

対象実験：direct_z_conv_delta_episode_v4 更新日：2026 年 9 月 23 日

1 目的と全体構成

本プログラムの現在の目的は、元エピソードから抽出した潜在変数を用いて、元データと似た特徴を持つ新しいエピソードを生成できるか確認することである。入力履歴に続く未来だけを予測する方式から、エピソード全体を入力・生成対象とする方式へ変更した。対象は単変量の RSRP 系列であり、通信制御 AI や物理シミュレータとの接続は現行処理に含まない。

1.1 全体のアーキテクチャ



破線は学習時に正解系列へノイズを加える経路を示す。生成時は**潜在変数 z** と**拡散ノイズ**だけを使い、元エピソードを生成器へ直接渡さない。同じ z からでも、ノイズを変えることで複数系列を生成する。

1.2 入出力と初期設定

項目	内容
入力データ	CSV の <code>rsrp</code> 列。単位 dBm。窓はファイル境界をまたがない。
エピソード長	$L = 40$ ステップ。約 2 Hz なら約 20 秒だが、実時間は元の測定間隔に依存する。
潜在変数	$D = 32$ 次元の実数ベクトル。Encoder の出力そのもの。
生成数	標準評価は元エピソードごとに $S = 32$ 系列。
出力データ	生成系列・元系列の CSV、潜在変数ファイル、評価 JSON、比較図。
対象外	多変量観測、可変長系列、基地局操作による状態遷移の計算。

B はバッチ内のエピソード数である。現在は通常・危険の全窓を扱い、危険だけへの自動抽出や危険例生成率による評価は行わない。

2 データ処理とモデルの内部構造

2.1 窓の作成と標準化

RSRPEpisodeDataset はファイルごとに長さ L の連続窓を 1 ステップ刻みで作る。長さ N のファイルでは $N - L + 1$ 個の窓が得られる。短すぎるファイルは窓作成対象から除かれる。入力是有限値の RSRP 系列を想定し、非有限値があればエラーとする。時刻の補間を行わないため、時刻間隔や欠測の処理は前処理側で確認する必要がある。

学習用の各系列を時間順に前方 80%・後方 20% へ分け、それぞれ学習・検証に使う。境界をまたぐ窓は除外する。正規化の平均 m_{tr} と標準偏差 s_{tr} は **検証部分を除いた学習区間のみ**から求める。

$$x_{0,\ell} = \frac{r_\ell - m_{\text{tr}}}{s_{\text{tr}}}, \quad \tilde{r}_\ell = s_{\text{tr}} \tilde{x}_{0,\ell} + m_{\text{tr}}.$$

標準偏差には数値安定性のため下限 10^{-6} を置く。元エピソードごとの平均値や最終値を生成結果へ加算する処理はない。最終評価には別途指定された評価用ファイルを用いる。

2.2 Encoder：全体系列から直接 z を得る

処理	内容	標準のテンソル形状
入力射影	RSRP の 1 次元を 128 次元へ線形変換	$[B, 40, 1] \rightarrow [B, 40, 128]$
位置情報	正弦・余弦による位置符号化を加算	$[B, 40, 128]$
Transformer	4 ヘッド、3 層の Encoder	$[B, 40, 128]$
代表特徴の抽出	最後のトークンの出力を使用	$[B, 128]$
潜在射影	線形層と SiLU	$[B, 128] \rightarrow [B, 32]$

Attention には因果マスクを用いず、最後のトークンの表現もエピソード全体を参照できる。最後の RSRP 値だけを使うという意味ではない。

$$z = E_\phi(x_0)$$

$\mu \cdot \sigma$ の推定、潜在変数の確率的サンプリング、KL 正則化は行わない。学習時には dropout (設定値 0.05) を用いるが、推論時に `model.eval()` とすると、同じ入力から同じ z を得る。各潜在次元に遮蔽量などの物理的意味を指定してはいない。

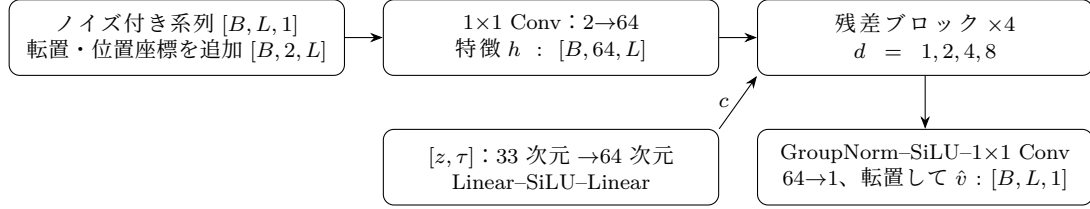
2.3 拡散モデル：ノイズ付き系列・拡散時刻・ z から v を予測

現在の予測器は TemporalConvPredictor であり、出力対象はノイズそのものではなく拡散モデルのパラメータ v である。この v は RSRP の時間差分や移動速度を意味しない。ノイズ付き系列を $[B, 1, L]$ へ変換し、位置座標 (−1 から 1) を追加する。1×1 畳み込みで 64 チャンネルへ変換し、4 個の時間方向残差ブロックへ渡す。各ブロックはカーネル幅 3 の畳み込みを 2 回使い、dilation を順に 1、2、4、8 とする。 z と拡散ステップの埋込みを各ブロックにスケール・シフトとして与える。GroupNorm・SiLU・1×1 畳み込みで $[B, L, 1]$ の v を出力する。畳み込み経路の受容野は 61 ステップであり、パディングによって系列長を保つ。全体生成用なので因果マスクは用いない。Encoder は引き続き同時学習する。各ブロックの残差接続は $(h + F(h, z, t))/\sqrt{2}$ である。ここで h は各時点の 64 次元特徴であり、元 RSRP を出力へ直接足してはいない。条件は正規化した特徴に $(1 + a(z, t))$ を掛け、 $b(z, t)$ を足して反映する。GroupNorm はチャンネルと時間方向を含めて正規化するため、61 ステップは畳み込み経路だけの受容野である。旧 MLP 方式は比較用設定 `configs/mlp_baseline.yaml` から利用できる。

3 追加解説：時間方向の畳み込み生成器

3.1 MLP から変更した部分と入出力

変更対象は拡散モデルの v 予測器であり、Transformer Encoder は維持する。旧 MLP は系列をまとめて全結合層へ入力したが、Conv1d は近傍時点に同じ学習重みを適用する。これにより、異なる位置に現れる局所的な変化を共通の処理で扱える。時系列の位置 i と、ノイズ除去の段階 t は別の変数である。



位置座標は先頭 -1 から末尾 1 までの固定値で、純粋なノイズからでも位置を区別する手掛かりとなる。 h_i は各時点の 64 個の内部特徴であり、RSRP そのものではない。 1×1 Conv は同じ時点のチャンネルを混合し、時間方向の混合は後続の幅 3 の Conv で行う。

3.2 畳み込みが参照する時点

幅 3 、参照間隔 d の畳み込みは、出力チャンネル o について次の計算を行う。

$$u_{o,i} = b_o + \sum_{c=1}^{64} \sum_{j=-1}^1 W_{o,c,j} h_{c,i+jd}.$$

$d=1$ なら $i-1, i, i+1$ 、 $d=4$ なら $i-4, i, i+4$ を参照する。重み W とバイアス b は学習され、時間位置によらず共有する。両端に幅 d のゼロパディングを加え、長さ L を維持する。各ブロックに幅 3 の Conv が 2 回あるため、畳み込み経路の受容野は $1 + 4(1 + 2 + 4 + 8) = 61$ ステップである。GroupNorm は各系列のチャンネル・時間全体を参照するため、これはモデル全体の依存範囲の上限ではない。

3.3 条件付けと残差接続の計算

コードの拡散添字を $q = 0, \dots, T-1$ とすると、入力する時刻は $\tau = q/(T-1)$ である。 $c = \text{MLP}([z, \tau])$ を共通条件とし、各ブロックが固有の線形層で $a, b \in \mathbb{R}^{64}$ を作る。 a, b は時間方向へ同じ値を配る。

$$\begin{aligned} u &= \text{Conv}_{d,1}(\text{SiLU}(\text{GN}_1(h))), \\ \tilde{u} &= (1 + a(c)) \odot \text{GN}_2(u) + b(c), \\ h_{\text{out}} &= [h + \text{Conv}_{d,2}(\text{SiLU}(\tilde{u}))]/\sqrt{2}. \end{aligned}$$

GN は $\text{GroupNorm}(1, 64)$ 、SiLU は非線形関数である。残差接続は入力特徴を後段へ伝え、 $1/\sqrt{2}$ は加算後の大きさの増加を抑える係数である。生成時はこの予測器を 100 回呼び出して逆拡散するが、学習時は各系列で選んだ 1 段階を処理する。畳み込みは固定の平滑化ではなく、鋭さの再現も保証しないため、差分分布で評価する。実装は `TemporalConvPredictor` と `TemporalResidualBlock` である。

4 学習・生成アルゴリズム

4.1 学習時の計算

数式では拡散ステップを $t = 1, \dots, T$ と表す（コードの配列添字は $0, \dots, T-1$ ）。 $T = 100$ で、累積信号係数 $\bar{\alpha}_T = 0$ になるようノイズスケジュールを再調整する。各エピソードについて t を一様に選び、独立な $\epsilon \sim \mathcal{N}(0, I)$ を生成する。

$$z = E_\phi(x_0), \quad (1)$$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad (2)$$

$$v_t = \sqrt{\bar{\alpha}_t} \epsilon - \sqrt{1 - \bar{\alpha}_t} x_0, \quad (3)$$

$$\mathcal{L}_v(\phi, \theta) = \mathbb{E}_{x_0, t, \epsilon} \left[\frac{1}{L} \|\hat{v}_\theta(x_t, t, E_\phi(x_0)) - v_t\|_2^2 \right]. \quad (4)$$

復元系列を $\hat{x}_0 = \sqrt{\bar{\alpha}_t} x_t - \sqrt{1 - \bar{\alpha}_t} \hat{v}$ として、隣接差分 $\Delta x_i = x_{i+1} - x_i$ の補助損失を加える。

$$\mathcal{L} = \mathcal{L}_v + \lambda_\Delta \mathcal{L}_\Delta, \quad \mathcal{L}_\Delta = \mathbb{E} \left[\frac{1}{L-1} \sum_{i=1}^{L-1} (\Delta \hat{x}_{0,i} - \Delta x_{0,i})^2 \right].$$

標準設定は $\lambda_\Delta = 0.1$ で、標準化した系列で計算する。追加の反復生成は不要で、検証の合計損失でモデルを選ぶ。損失は無次元の標準化空間で計算し、 $\lambda_\Delta = 0$ で従来の v 損失のみとなる。二階差分は評価用であり、現在の学習損失には含めない。正解の z を与える教師データはなく、生成に役立つ表現を学ぶよう、 z を通じて Encoder にも勾配を伝える。現在は入力と生成対象が同じエピソードである。モデル API には同じ環境条件から得た別エピソードを `target` として渡す機能もあるが、標準の学習ループでは使用していない。

学習設定	現在の設定値・処理
最適化	AdamW、学習率 10^{-4} 、バッチサイズ 64。
安定化	勾配ノルムの上限 1.0。
学習回数	最大 150 エポック、最低 30 エポック。
モデル選択	検証合計損失の実際の最小値を保存。
Early Stopping	0.001 を超える検証損失改善が 15 エポック連続でなければ、最低回数以降に終了。
再現性	学習 seed は 42。検証 seed は 12345 で毎回固定し、学習側の乱数状態を保存・復元。

ログには合計損失 `loss`、`diffusion_mse`、`delta_mse` を記録する。CRPS は診断用に記録するが、現行のモデル選択には使っていない。生成を伴う診断は、検証 seed で重複なしに選んだ固定の最大 256 エピソードに限定する。検証損失は全検証エピソードで計算する。生成本数は各 32 本、診断頻度は毎エポックのままである。独立した評価コマンドでは全評価エピソードを使用する。

5 生成処理と評価指標

5.1 生成時の計算

元エピソードを一度 Encoder へ入力して z を得る。各生成例の x_T は独立な標準正規ノイズから開始し、 z を固定して逆拡散する。

$$\hat{x}_0 = \sqrt{\bar{\alpha}_t} x_t - \sqrt{1 - \bar{\alpha}_t} \hat{v}_\theta(x_t, t, z), \quad (5)$$

$$x_{t-1} = c_t \hat{x}_0 + d_t x_t + \sqrt{\tilde{\beta}_t} \xi_t. \quad (6)$$

$c_t, d_t, \tilde{\beta}_t$ はノイズスケジュールから計算する DDPM の係数であり、 ξ_t は新しい標準正規ノイズである。最後のステップではノイズを加えない。完了後、学習時の統計量を用いて dBm へ戻す。初期ノイズと途中のノイズが異なるため、同じ z から複数の系列が得られる。

5.2 API と保存済み z の利用

```
model.eval()
z = model.encode(normalized_episode)          # [B,32]
x = model.generate(z, num_samples=32)         # [B,32,40,1]
rsrp_dbm = x * training_std + training_mean
```

保存済み z を使う場合は Encoder と元 CSV を参照せず生成できる。モデルと潜在ファイルの対応はチェックポイントの SHA-256 で照合する。旧確率潜在モデルの重み・潜在ファイルとは非互換である。

5.3 評価指標の定義

元系列の値を $y_{n\ell}$ 、生成値を g_{nsl} とし、 n をエピソード、 $s = 1, \dots, S$ を生成例、 ℓ をステップとする。生成平均は $\bar{g}_{n\ell} = S^{-1} \sum_s g_{nsl}$ である。

$$\text{MAE} = \frac{1}{NL} \sum_{n,\ell} |\bar{g}_{n\ell} - y_{n\ell}|,$$
$$\text{CRPS} = \frac{1}{NL} \sum_{n,\ell} \left[\frac{1}{S} \sum_s |g_{nsl} - y_{n\ell}| - \frac{1}{2S^2} \sum_{s,r} |g_{nsl} - g_{nr\ell}| \right].$$

いずれも RSRP の差に基づく量で、小さいほど元系列に整合する。

その他の指標	定義・注意点
90% 区間被覆率	各点の元の値が生成値の 5–95% 分位区間に入る割合。区間幅と併せて評価する。
ステップ別統計	CRPS、MAE、平均誤差、元・生成の標準偏差、被覆率、区間幅。
固定 z 内の標準偏差	同じ元エピソード・同じステップの生成例間標準偏差を、全エピソード・ステップで平均する。

これらは元エピソード全体を Encoder に与えた条件付き再生成の評価であり、独立した未来予測の精度ではない。生成のばらつきが大きければよいわけではなく、過大な裾や区間幅にも注意する。危険例生成率は現段階の指標に含まない。

6 時間的特徴の評価とグラフ

再生成の平均誤差だけでは、個々の生成系列が持つ揺れや鋭い変化を評価できない。以下は逆標準化した各系列 $r_1, \dots, r_L$ について計算する。元系列・生成系列とも全評価エピソードを用い、生成系列の中央値に置き換えない。

6.1 一階・二階差分の分布

$$\Delta r_i = r_{i+1} - r_i, \quad \Delta^2 r_i = r_{i+2} - 2r_{i+1} + r_i.$$

一階差分は隣接時点の上昇・下降の大きさ、二階差分は傾きの切り替わりを表す。各系列内で計算し、境界をまたぐ差分は含めない。単位はいずれも dB で、時間刻みで割った微分ではない。共通の 100 ビンに全値を集計し、面積 1 の確率密度として比較する。外れ値は除外しない。分布が一致しても変化の順序や発生位置が同じとは限らない。

- `episode_delta_distribution.png` : 一階差分。元系列 $N(L-1)$ 個、生成系列 $NS(L-1)$ 個。
- `episode_second_delta_distribution.png` : 二階差分。元系列 $N(L-2)$ 個、生成系列 $NS(L-2)$ 個。

緑が元系列、青が生成系列である。凡例には個数と標準偏差を表示する。

6.2 自己相関と平均二乗変化量

自己相関は各系列の平均 $\bar{r}$ を引き、全系列の偏差平方和を分母に固定する。

$$A(k) = \frac{\sum_{i=1}^{L-k} (r_i - \bar{r})(r_{i+k} - \bar{r})}{\sum_{i=1}^L (r_i - \bar{r})^2}, \quad D(k) = \frac{1}{L-k} \sum_{i=1}^{L-k} (r_{i+k} - r_i)^2.$$

$A(k)$ は時間的な持続性、 $D(k)$ は時間差ごとの変動の大きさを表す。 $k = 0, \dots, \lfloor L/2 \rfloor$ とし、標準設定では 0-20 ステップを表示する。 $D(k)$ の単位は dB^2 で、元系列と生成系列の点ごとの MSE とは異なる。

- `episode_autocorrelation.png` : 自己相関 $A(k)$ 。定数系列は未定義として除外し、その数を明記する。
- `episode_mean_squared_change.png` : 平均二乗変化量 $D(k)$ 。定数系列も含める。

各エピソードで計算した後、線で平均、帯で系列間の 10-90% 点を表示する。帯は平均の信頼区間ではない。窓が重なる元系列や同じ z の生成例は独立標本ではない。自己相関は振幅を正規化するため、 $D(k)$ や差分分布と併せて読む。

6.3 元系列と生成例の折れ線比較

`episodes.png` は、元エピソード 1 本を無作為に選び、対応する生成例から重複なしに 8 本を選ぶ。2 列 $\times$ 4 行の各パネルに同じ元系列（緑）と生成例 1 本（青）を重ねる。選択は評価 seed で再現でき、生成本数が 8 未満の場合は全本数を表示する。この図は 1 つの潜在状態についての比較であり、全体の代表性を保証しない。`episode_boxplot.png` は元・生成の全値の箱ひげ図である。箱は 25-75% 点、線は中央値、ひげは 1.5 IQR 以内の最遠点であり、外れ値は非表示である。ひげを最大・最小値と解釈しない。ステップ別箱ひげ図と生成中央値の箱ひげ図は出力しない。

7 保存物・実行手順・研究上の位置付け

7.1 ファイル入出力

保存物	内容
生成 CSV	episode_id, sample_id, episode_step, rsrp_generated_dbm. 1 行が 1 生成例の 1 時点。
元系列 CSV	episode_id, episode_step, rsrp_actual_dbm。
潜在ファイル	CPU 上の z 、元 CSV 一覧、窓の位置、seed、モデル形式、チェックポイントの SHA-256。
チェックポイント	モデル重み、設定、学習の標準化統計量、最良エポック、検証指標、固定検証部分集合のインデックス。
学習ログ	training_metrics.json。各エポックの学習・検証損失、生成指標、評価対象本数。
評価 JSON	episode_metrics.json。再生成指標とステップ別統計。

標準の保存先は次のとおりである。

図・指標：
results/figures/direct_z_conv_delta_episode_v4/
重み：
results/checkpoints/direct_z_conv_delta_episode_v4/episodicdt.pt

生成 CSV 等は results/generated/ に保存する。実験名は v4 だがモデル構造は v3 と同じなので、内部の形式識別子は変更していない。識別子は direct_z_conv_episode_v3 である。旧設定で差分損失の重みが省略されていれば 0 として扱う。

7.2 実行方法とコードの対応

プロジェクト EpisodicDT-small 直下で実行する。

```
conda activate episodicdt
python -m src.train.train --config configs/config.yaml
python -m src.evaluation.evaluate --samples 32 --seed 12345
# 保存済み $z$ からの生成（各パスは実際の保存先を指定）
python -m src.evaluation.generate --checkpoint MODEL.pt \
    --latents LATENTS.pt --output generated.csv --samples 16
```

src/data/dataset.py がデータ、src/models/episodic_diffusion.py がモデル、src/train/が学習・検証、src/evaluation/が評価・再生成を担当する。設定は configs/config.yaml で管理する。学習・評価・保存済み z からの生成は CUDA が利用可能なら GPU を使い、それ以外は CPU を使う。評価結果はバッチごとに CPU へ戻して集計・保存する。学習中の生成評価のみ最大 256 本に限定し、評価コマンドは全評価窓を対象にする。

7.3 現在の限界と研究への位置付け

小さい再生成誤差だけでは、元データの記憶やコピーを排除できない。潜在変数の交換実験、最近傍系列との比較、時間構造・分位点の評価を合わせて行う必要がある。物理的な潜在状態との対応や生成例の実現可能性も未検証である。最終的には潜在表現からシミュレータの環境条件を生成し、制約と実測整合性を確認した上で、そのシナリオを用いた制御 AI 学習の効果を独立したシミュレーション条件で評価する。

7.4 Overleaf でのコンパイル

同梱 ZIP を新規プロジェクトとしてアップロードし、主ファイルを main.tex、コンパイラを XeLaTeX に設定する。構造図は TikZ で描くため外部画像は不要である。日本語フォントは Noto CJK があれば使用し、なければ TeX Live の原ノ味フォントを使用する。pdfLaTeX は使用しない。